

近似算法

Alan_Zhao

2026 年 5 月 4 日

目录

- 1 作为热身的一些例子
- 2 无权近似 APSP
- 3 线性规划与近似算法
 - 集合覆盖 once again
 - 线性规划对偶
 - 集合覆盖 the third take
 - 近似 Steiner 森林
 - 後日談

近似算法是什么?

对于最优化问题而言, 一个可以求出一个近似最优的解的算法就可以被称作近似算法.

计数问题也有近似算法, 但是这里不会涉及.

一般来说, 我们会分析一个近似算法的**近似比**. 对于最小化某个目标的问题, 假设最优解是 A , 算法求出的解是 A' , 我们希望设计一个算法, 使得最坏情况下的 A'/A 尽量小.

对于一些特殊的问题, 我们可以做到**加性近似**, 即 $A' - A$ 可以被某些式子 bound 住.

今天会讲什么?

主要是两部分: 第一部分是一些具体的近似算法例子, 第二部分是线性规划与近似算法的联系.

不会讲具体求解线性规划的算法, 事实上我们只需要用到线性规划的形式, 而不需要用到具体的算法.

热身 1

负载均衡问题: 有 n 个任务和 m 台机器, 第 i 个任务需要 t_i 的时间去做, 每个机器在同一时刻只能做单个任务, 如何给每个任务分配机器, 使得所有机器的结束时间的最大值最小?

要求: 1.5-近似.

热身 1: Solution

按照时间递减顺序分配. 每次把耗时最大的未分配任务分配给当前任务总量最少的机器.

考虑最晚完成的机器, 设分配给它的最后一个任务时间为 t_n . 在分配该任务之前, 这台机器的负载不超过平均值, 即不超过 $\frac{1}{m} \sum t_i \leq T^*$.

只考虑 $n > m$ 的情况, 此时前 $m+1$ 个任务中必然至少有两个被分配到同一台机器, 则有 $T^* \geq 2t_{m+1}$. 又因为任务按耗时递减, 所以 $t_n \leq t_{m+1} \leq \frac{1}{2}T^*$. 于是最晚完成的机器总时间 $T \leq T^* + t_n \leq 1.5T^*$.

热身 2

集合覆盖问题: 给定一个元素集合 U 和一些 U 的子集 $S_1, S_2, \dots, S_m$, 每个子集 S_i 有一个权重 w_i , 选择一些子集使得它们的并集是 U , 并且这些子集的权重之和最小.

要求: $\ln n$ -近似.

热身 2

集合覆盖问题: 给定一个元素集合 U 和一些 U 的子集 $S_1, S_2, \dots, S_m$, 每个子集 S_i 有一个权重 w_i , 选择一些子集使得它们的并集是 U , 并且这些子集的权重之和最小.

要求: $\ln n$ -近似.

提示: 猜猜 $\ln n$ 是从哪里冒出来的.

热身 2: Solution

贪心策略: 每次选择 w_i /新覆盖元素数 最小的集合 S_i 加入.

定义 c_x 为元素 x 首次被覆盖时, 产生均摊花费 $c_x = w_i$ /新覆盖元素数.
显然有 $\sum w_i = \sum c_x$.

考虑最优解选出的任意集合 S^* , 假设当前 S^* 中还有 k 个元素未被覆盖.
此时我们完全可以选择整个 S^* , 它的性价比是 w_{S^*}/k . 于是新覆盖的元素 x 必然满足 $c_x \leq w_{S^*}/k$.

所以 $\sum_{x \in S^*} c_x \leq H_{|S^*|} \cdot w_{S^*}$. 对最优解的所有集合求和, 可知这达到了 $\ln n$ 的近似界.

热身 3

全源最短路 (APSP): 给定无向无权图 G , 求出所有点对之间的最短路长度.

要求: $+2$ -近似, 时间复杂度 $\tilde{O}(n^{2.5})$.

热身 3

全源最短路 (APSP): 给定无向无权图 G , 求出所有点对之间的最短路长度.

要求: $+2$ -近似, 时间复杂度 $\tilde{O}(n^{2.5})$.

提示: 有没有一张比较稀疏的子图, 使得其最短路与原图最短路相差不大?

热身 3: Solution

随机选择一个大小为 $\tilde{O}(\sqrt{n})$ 的 hitting set D . 可以保证图中所有度数 $\geq \sqrt{n}$ 的点都会与 D 中某个点相邻.

将查询转化为在更小的图上跑 BFS. 我们保留以下部分:

- 1 所有与度数 $< \sqrt{n}$ 点相邻的边, 条数 $O(n^{1.5})$.
- 2 从 D 中每个点开始向整张图跑单源最短路形成的树边, 条数 $\tilde{O}(n^{1.5})$.
- 3 连接 D 与所有大度数点的跨越边, 条数 $O(n)$.

最终拼出的图边数为 $\tilde{O}(n^{1.5})$, 在其上跑全源最短路即可得到 +2-近似, 总时间 $\tilde{O}(n^{2.5})$.

无权近似 APSP: $\tilde{O}(n^{7/3})$

- 根据度数将点集分为三级 $S_0 \supset S_1 \supset S_2$, 其中 S_i 是所有度数至少为 $n^{i/3}$ 的点.
- 对应的 hitting set 设为 D_i , 大小为 $\tilde{O}(n^{1-i/3})$.
- 如果 $u \rightarrow v$ 的最短路上没有 $S_1 \setminus S_2$ 中的点, 直接沿用上述做法.
- 否则, 考虑最短路上最后一个属于 $S_1 \setminus S_2$ 的点, 设与它相邻的 hitting point 是 x , 那么 $u \rightsquigarrow x$ 和 $x \rightsquigarrow v$ 两部分路径可以分开讨论.

无权近似 APSP: $\tilde{O}(n^{7/3})$

- 根据度数将点集分为三级 $S_0 \supset S_1 \supset S_2$, 其中 S_i 是所有度数至少为 $n^{i/3}$ 的点.
- 对应的 hitting set 设为 D_i , 大小为 $\tilde{O}(n^{1-i/3})$.
- 如果 $u \rightarrow v$ 的最短路上没有 $S_1 \setminus S_2$ 中的点, 直接沿用上述做法.
- 否则, 考虑最短路上最后一个属于 $S_1 \setminus S_2$ 的点, 设与它相邻的 hitting point 是 x , 那么 $u \rightsquigarrow x$ 和 $x \rightsquigarrow v$ 两部分路径可以分开讨论.

不断重复这个结构, 可以做到 $\tilde{O}(k \cdot n^{2+1/k})$ 的 $+2(k-1)$ 近似.

集合覆盖 once again (I)

集合覆盖可以写作整数规划的形式.

最小化 $\sum_{i=1}^k c(S_i)x_i$, 约束条件

$$\begin{cases} \sum_{i:e \in S_i} x_i \geq 1, \forall e \in U, \\ x_i \in \{0, 1\}. \end{cases}$$

要求做到 f -近似, 其中 f 是每个元素被覆盖的集合数的最大值.

集合覆盖 once again (II)

众所周知, 整数规划是 NP-hard 的, 但是线性规划不是. 它们的唯一区别就是是否要求 x_i 是整数.

那么, 如果求出线性规划的最优解, 我们也可以通过**取整**得到一组整数解.

在这个题中, 直接将所有 $x'_i \geq 1/f$ 的集合 S_i 取整为 1, 其他取整为 0, 这样就是一组 f -近似.

集合覆盖 once again (III)

但是, 既然最后得到了整数解, 我们一定需要解这个线性规划吗?
我们需要在不解线性规划的情况下, 分析线性规划的性质.

线性规划对偶 (I)

设 x, c 是 n 维向量, b 是 m 维向量, A 是 $m \times n$ 矩阵, 要最小化 $c^T x$, 约束条件 $Ax \geq b, x \geq 0$.

其对偶形式是最大化 $b^T y$, 约束条件 $A^T y \leq c, y \geq 0$.

线性规划对偶 (II)

将 y_i 理解为第 i 个约束条件 $\sum_{j=1}^n A_{ij}x_j \geq b_i$ 的系数, 将所有约束条件加起来, 得

$$\left(\sum_{i=1}^m y_i A_{i1}\right) x_1 + \cdots + \left(\sum_{i=1}^m y_i A_{in}\right) x_n \geq \sum_{i=1}^m y_i b_i. \quad (1)$$

要最小化的是

$$c_1 x_1 + \cdots + c_n x_n,$$

只要 $\sum_{i=1}^m y_i A_{ij} \leq c_j$, 即 $A^T y \leq c$, 就可以将最大化目标变为 $b^T y$. 当然 (1) 式需要取等才行, 这是由接下来的互补松弛条件保证的.

线性规划对偶 (III)

定理 (互补松弛条件)

x, y 分别是原问题和对偶问题的最优解, 当且仅当:

- 原始条件: $\forall j$, 要么 $x_j = 0$, 要么 $\sum_{i=1}^m A_{ij}y_i = c_j$.
- 对偶条件: $\forall i$, 要么 $y_i = 0$, 要么 $\sum_{j=1}^n A_{ij}x_j = b_i$.

这个定理让我们得以绕过线性规划的求解, 写出一些组合算法, 称作**原始-对偶算法**.

集合覆盖 the third take (I)

对偶, 要最大化 $\sum_{e \in U} y_e$, 约束条件

$$\begin{cases} \sum_{e: e \in S_i} y_e \leq c(S_i), \forall i, \\ y_e \geq 0, \forall e \in U. \end{cases}$$

设选择的集族是 $\mathcal{I}$. 根据互补松弛条件, 某个集合 $S_i \in \mathcal{I}$, 即 $x_i = 1$, 可以推出 $\sum_{e \in S_i} y_e = c(S_i)$.

我们的思路是, 同时维护 x, y , 只考虑互补松弛条件中的原始条件, 逐渐算出一组合法解, 然后证明最终得到的解是一组近似解.

集合覆盖 the third take (II)

算法

- 设 e 是一个没有被任何 $S_i \in \mathcal{I}$ 覆盖的元素.
- 增加 y_e , 直到存在一个 k 使得 $\sum_{e' \in S_k} y_{e'} = c(S_k)$.
- 将 S_k 放入 $\mathcal{I}$.

最终的 $\mathcal{I}$ 满足 $\forall S_k \in \mathcal{I}, \sum_{e \in S_k} y_e = c(S_k)$, 所以

$$c(\mathcal{I}) = \sum_{S_j \in \mathcal{I}} c(S_j) \leq f \cdot \sum_{e \in U} y_e.$$

但是对偶问题的解小于等于原问题的任意实数解, 于是 $c(\mathcal{I}) \leq f \cdot \text{OPT}$, 这是一个 f -近似算法.

集合覆盖 the third take (III)

我们成功用「组合」算法打平了线性规划!

接下来是一个线性规划算法无法处理的例子.

网络设计问题

有无向图 $G = (V, E)$, 每条边有代价 $c: E \rightarrow \mathbb{R}^+$. 给定一些对连通性的要求, 求代价最小的满足要求的子图.

热身: Steiner 树

给定 $T = \{t_1, \dots, t_k\} \subseteq V$, 要求出代价最小的将 T 连通的子图.

要求做到 2-近似.

提示: 这可以是 OI 题.

热身: Steiner 树

给定 $T = \{t_1, \dots, t_k\} \subseteq V$, 要求出代价最小的将 T 连通的子图.

要求做到 2-近似.

提示: 这可以是 OI 题.

建出一个以 T 为点集的完全图 H , 两个点 t_i, t_j 间的边权是在原图上的距离, 跑出 H 的最小生成树作为近似答案.

Steiner 森林 (I)

给定 k 个不交的集合 $T_1, \dots, T_k \subseteq V$, 要输出代价最小的子图 H , 使得每个 T_i 在 H 中都是连通的.

要求做到 2-近似.

Steiner 森林 (II)

将连通性的限制写作「割」的形式, 对于一个点集 S , 如果 S 和 $V \setminus S$ 将某个 T_i 分成了两部分, 则定义 $f(S) = 1$, 否则 $f(S) = 0$.

写出线性规划: 最小化 $\sum_{e \in E} c(e)x_e$, 限制条件

$$\begin{cases} \sum_{e \in \delta(S)} x_e \geq f(S), \forall S \subseteq V, \\ x_e \geq 0, \forall e \in E. \end{cases}$$

对偶: 最大化 $\sum_{S \subseteq V} f(S)y_S$, 限制条件

$$\begin{cases} \sum_{S: e \in \delta(S)} y_S \leq c(e), \forall e \in E, \\ y_S \geq 0, \forall S \subseteq V. \end{cases}$$

Steiner 森林 (III)

我们仍然只考虑原始条件, 即 $\forall e \in E, x_e = 0$ 或 $\sum_{S:e \in \delta(S)} y_S = c(e)$.

算法

- 设当前选中的边集是 F . 如果一个集合 $S \subseteq V$ 满足 $f(S) = 1$ 且 S 在 F 中的边构成的图上是连通分量, 则认为 S 是 active 的.
- 重复如下过程, 直到没有 active set: 同时增加当前所有 active sets 的 y_S , 直到存在 $e \in E$ 使得 $\sum_{S:e \in \delta(S)} y_S = c(e)$. 将 e 加入 F .
- 按照加入顺序的倒序考虑每条边 $e \in F$, 如果 $F \setminus \{e\}$ 是合法解, 则删去 e .

Steiner 森林 (IV)

只需要证明 $\sum_{e \in F} c(e) \leq 2 \sum_{S \subseteq V} y_S$, 这等价于

$$\sum_{S \subseteq V} y_S \cdot \deg_F(S) \leq 2 \sum_{S \subseteq V} y_S,$$

其中 $\deg_F(S)$ 是 S 在边集 F 中的出边条数.

Steiner 森林 (V)

考虑每次增加 active sets 的 y_S 对 LHS 和 RHS 的影响, 设当前 active sets 的集族是 $\mathcal{I}$, 实际上要证的是

$$\sum_{S \in \mathcal{I}} \deg_F(S) \leq 2|\mathcal{I}|.$$

由于 $\mathcal{I}$ 在 F 这个森林中是若干个不交的连通块, 并且所有能够对某个 $\deg_F(S)$ 造成贡献的边 $e \in F$, 它的两端的连通块中一定都有某个 $S \in \mathcal{I}$ (否则 $F \setminus \{e\}$ 是合法解, 所以 e 会被删去), 于是能够造成贡献的 e 都是 $\mathcal{I}$ 构成的「虚树」上的边, 得证.

後日談

费用流也有一个叫作原始-对偶的算法, 每个点的「势能」就是对偶变量. 因为费用流的约束矩阵是全幺模的, 所以原始-对偶算法得到的整数解是最优解.

後日談

费用流也有一个叫作原始-对偶的算法, 每个点的「势能」就是对偶变量. 因为费用流的约束矩阵是全幺模的, 所以原始-对偶算法得到的整数解是最优解.

近似算法不一定只能用来求近似解. 在一些问题里, 可以先求出一组近似解, 证明这组解与最优解相差不大, 然后用一些暴力算法增广到最优解. 例如, 二分图和一般图最大权完美匹配可以做到 $O(m\sqrt{n}\log(nW))$, 这里根号就是平衡原始对偶与增广算法的结果, 而 $\log(nW)$ 来源于 scaling 技术, 用来控制增广算法的迭代次数.